

当代人工智能：多模态情感分析 实验报告

10211900416 郭夏辉

实验截止时间：2024年1月31日23:59

项目的GitHub地址：https://github.com/tom6m6/Multimodal_Sentiment_Analysis

在完成这个实验的过程中，我花费了大量的精力——去调试代码、**研究论文**、**补充理论知识**、分析结果。报告的结构可能并不是那么严谨，而是通过很多“自问自答”来阐释实验的步骤。

1.正式实验之前

1.1 这个实验的目标是什么？

给定配对的文本和图像，预测对应的情感标签。三分类任务：positive, neutral, negative。

我要设计一个多模态融合模型，这个模型能够整合输入图片和文本数据输出其情感标签。在此模型设计出来之后，我要自行从训练集中划分验证集、调整超参数，然后在测试集上预测情感标签。虽然这里输入数据是多模态的，但最终解决的本质问题还是**多分类任务**。

1.2 数据集的样貌是什么样子的？

我粗略看了一下老师给出的数据，感觉这个数据集大概是来源于社交媒体。每对图像-文本**并不是那么规整**——图像的大小和主题都是不固定的；文本是多语言的，甚至还有很多不规范用语、社交媒体特性（#标签 @某人）

在给定的训练数据中，还有一个值得注意的特征是类别并不均衡。positive 占多数，**neutral 占比很少**——2388对positive，1193对negative，仅419对neutral。

有时候，单从一个模态分类是比较容易的，我们之前的几次实验就是这样做的；但更多的时候，这样做是比较困难的。结合两种模态的特征进行学习，我们可以提取到更全面的信息，往往能取得较好的预测结果。但假设我们对配对内容没有先验知识，只是从表面意思上来理解文本和图像，忽视了更重要的隐藏信息，这样多模态也不一定能取得准确的结果了。

数据的真实带来了很强的不规则性，给这个分类任务增添了不少的难度，我感觉自己不太能够在此背景下取得非常精确的结果。（比如前几次实验时那样动辄90%+的验证集准确率）

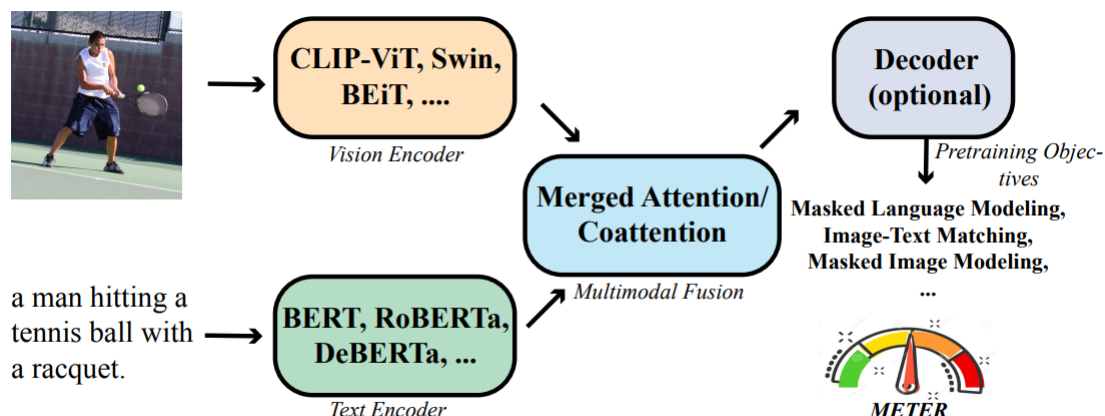
1.3 多模态分类任务的基础架构是什么？

关于最基础的知识，老师已经在课堂中讲到了，我再在有限的报告中长篇地回忆一遍显然不现实。本人觉得这篇博文写的不错¹，可以帮助我们快速回忆一下相关的理论。

1.3.0 VLP任务概览

我在网上找了不少相关的论文，觉得这篇论文²写得还是比较好的，我也主要总结一下这篇论文中介绍到的理论。

VLP (Visual-Language Pretraining) **一般具有五个模块**，即输入图片和文本对，图片经过视觉编码器 (CLIP-ViT, Swin, BEiT等) 编码，文本经过文本编码器 (BERT, RoBERTa, DeBERTA等) 编码，之后两者经过多模态融合模块 (Merged Attention/Co-Attention) 进行模态信息交互，最后经过一个可选的解码器输出结果：



作者结合着近几年主流VLP模型，将其大致分为三类，并且总结了一下它们各自的特点：

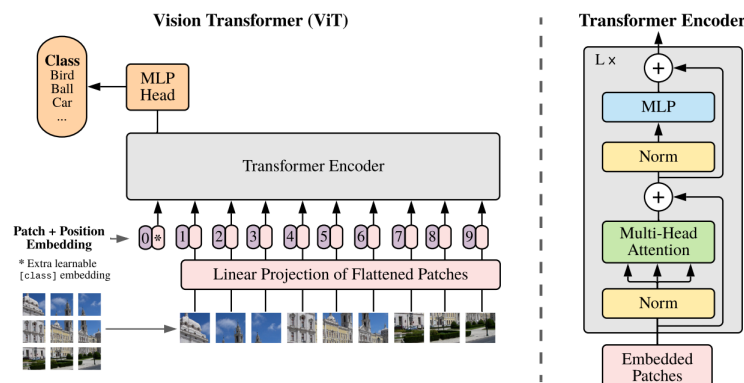
- OD-based Region Features 使用object detector来提取视觉特征。有些模型也可以**分别处理图像及文本特征，再attention合并**。缺点在于预训练时OD被冻结了，而且非常消耗时间。
- CNN-based Grid Features 中，以Pixel-BERT和CLIP-ViL为例，它们使用传统CNN网络得到 grid features，文本使用transformer；SOHO先使用可学习的视觉词典离散化grid features，然后送入多模态模块中。在这个过程中，可能优化器不一致，比如CNN使用SGD，transformer使用AdamW，并且效果来看它并不是那么优秀。
- ViT-based Patch Features **直接使用ViT处理patch features**，如ALBEF。虽然它在VQA等下游任务上表现不佳，但它总体上还是被证明取得了不错的结果。

在本次实验中我对预训练的CLIP做出选择时，据此选择的是ViT-based Patch Features，当然这就是后话了。

除了最开始的输入模块，我重点要来介绍一下VLP模型的**其他四个模块**，它们也是模型的主要组件。我来以这篇论文为主线，参阅其他一些论文——来看：

1.3.1 Vision Encoder

这篇仅关注基于transformer的视觉编码器，使用的是patch features，之前的一些VLP直接使用的ViT，文章考虑了很多种编码器，但是我主要还是来讨论ViT。ViT的结构在另一篇论文中³有所阐释：



我也看了一下这篇博文⁴与这个论文逐句讲解⁵。概括来说是如下几点：

- ViT首先将2D图像转换成1D patch embeddings，除此之外还要进行一下position embedding。
- 虽然图像的信息是二维的，但实际发现**1D positional embedding**效果和2D不存在显著的差异。
- 经过Transformer Encoder之后，**ViT借鉴BERT增加一个MLP作为分类头**，原来的sequence也因此需要多一项class token。
- 依托Transformer强大的可拓展性，ViT在数据规模较大的情况下表现十分出色。但是**数据规模较小**的情况下这样的结构反而可能不是CNN的对手。

在这篇论文中⁶，Kaiming He 等人通过残差连接这一方法，解决了网络层数加深导致的特征消失问题，并实现了最深 152 层的神经网络。关于ResNet的技术细节和优点，它的核心在于**残差块**，我在此就不多说了。

在后续的实验中，应用普通的架构时，我使用的是**ResNet-50**模型作为图像特征提取器，之所以选择它的核心原因在于它的网络深度比较大，对图像特征的提取理论上来说应该有还不错的效果。但是我使用的这个ResNet-50，也不是像第三次实验时那样自己从头到尾训练的，而是在ImageNet上面**已经预训练**好的，这可以在大幅减少本机训练的时间。

1.3.2 Text Encoder

预训练模型的使用是当今NLP工作的主流，其中**BERT**模型又是预训练方法的代表。BERT是一个自编码语言模型，预训练的过程特别像我们做“完型填空”。它的主体由多个Transformer结构组成，网络首先接受一串由词项和特殊标识符（CLS、SEP）经过嵌入得到的句子向量，并使用Transformer构架计算句子的语言特征，得到一串同等长度的语言特征向量。在连接网络的输出与下游的网络之后，我们就可以进行各种NLP任务。对于当前文本分类任务，我们只需要使用网络输出中的第一项（CLS标识符的特征表示），并将其连接至一个全连接层即可得到文本的概率分类结果。

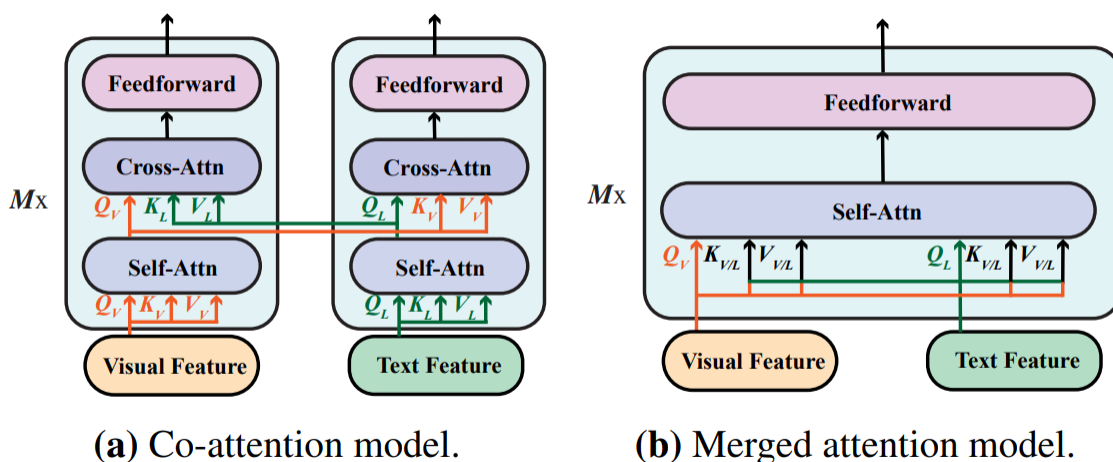
在传统的BERT基础上，**RoBERTa通过改进预训练任务**，进一步**增强了语言特征的提取能力**，具体的提升在这篇论文⁷里有所阐述。我在后续的实验过程中，应用普通的架构时，使用的是Huggingface的transformers之RoBERTa预训练模型作为文本特征提取器。

1.3.3 Multimodal Fusion

现在到了模型融合的阶段了，我觉得这是这篇论文讲得很好的地方。由于不同模态对于信息的提取都是局部的，彼此之间需要一些补充、去重、对比才能形成完整的认知，所以我们有必要对模态进行融合。

论文介绍了两种融合方式：**Co-attention**和**Merged attention**。

- Co-attention: 文本和视觉特征送入**各自的transformer block**，然后cross attention一下，信息就彼此相互传递了，参数较多，也较为复杂。
- Merged attention: 文本和视觉特征简单的进行拼接，送入**一个transformer block**中。



值得一提的是，我在本次实验中尝试的融合方法都是无权重的——元素相加、向量拼接的过程**并没有乘一个权重**，这样也许能避免一些算法歧视问题吧。

1.3.4 Pretraining Objectives

许多VLP模型**只使用encoder**

预训练任务一般被分为如下几类：

- **Masked Language Modeling.** 随机将文本中的token遮蔽，结合未遮蔽的文本信息和视觉信息进行重建。比如说BERT那样的完型填空
- **Image-Text Matching.** 一个batch中有match的图文对和不match的图文对，模型进行二元分类，通常使用[CLS] token进行分类。比如CLIP最大化对角线的正样本相似度，最小化其余负样本相似度
- **Masked Image Modeling.** 类似于MLM，LXMBERT和UNITER将输入中的一些regions遮蔽，预测遮蔽regions的feature，是回归任务，使用MAE loss。部分研究预测遮蔽regions的object labels，是分类任务，使用交叉熵损失。

1.3.5 重要结论总结

当然我觉得作者最主要的是提出了如下几个结论：

- **VLP模型encoder-only比使用encoder-decoder的效果好。** 只有encoder的VLP模型在VQA和zero-shot图文检索任务上表现更好。
- **模态之间有cross-attention比只使用self-attention效果更好**
- **MIM(Masked Image Modeling)在VLP中对于模型的性能是没有帮助的**
- 视觉transformer比文本transformer发挥了更大的作用
- ViT在VLP中扮演者更重要的角色，**在纯视觉和纯文本任务上的性能不能很好的反映VL任务。**

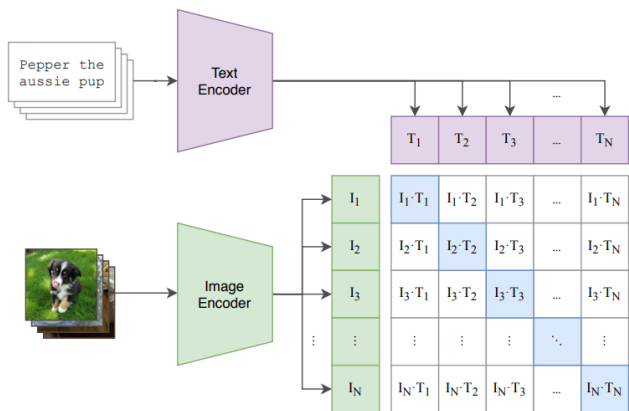
1.4 有没有超脱于基础架构，表现还更好的模型？

在实验报告的1.3部分，我介绍了一下多模态分类任务的基础架构。但我感觉这样还是不够的，我想找一个效果更好的模型。彼时恰逢本人刚刚开始接触科研，看到了这个论文解析播放列表⁸，然后我看到了这篇论文⁹的讲解，在此之中，我了解到了CLIP这种基于**对比学习**的图像-文本多模态模型。它可以同时理解图像和文本，只需要一个简单的线性分类器就可以完成相应任务，**甚至不需要微调**。它的**迁移泛化能力强**，作为预训练模型，跟特定任务解耦(task-agnostic)，可以实现类似**zero-shot**的效果。经过一顿安利（忽悠），这个模型深深地吸引了我，让我花了很多时间去拜读它的原始论文¹⁰。

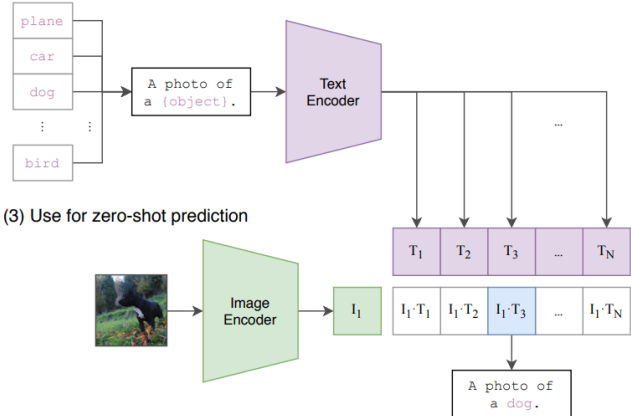
1.4.1 CLIP概览

CLIP的核心思想是用图像和文本之间的语义相似性来训练一个视觉-语言模型。它是OpenAI的第一篇多模态预训练的算法，延续了GPT系列“大力出奇迹”的传统。OpenAI采集了超过4亿的图像-文本对，通过Text Encoder和Image Encoder分别得到了文本和图像的特征。然后，使用了一个对比损失函数，优化两个表征所对应的**向量的相似度**——最大化对角线的正样本相似度，最小化其余负样本相似度。它通过对比学习来进行模型训练，训练的过程详见论文。在如此庞大的数据训练基础之上，CLIP学习到了一个**通用的视觉-语言表示**。

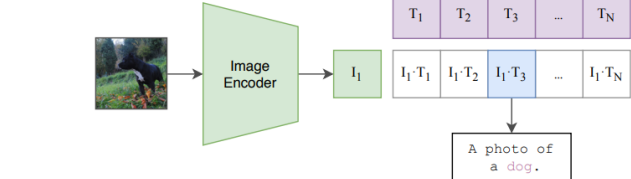
(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



训练完成后，CLIP就可以用来做各种视觉和语言任务了。如果我们要用CLIP来分类图像，只需要给出一些自然语言的标签，例如“猫”、“狗”等，然后它就可以将每个标签转化为一个表征，计算它和图像表征之间的内积，选择内积最大的标签作为图像分类的结果。

在论文中，介绍了CLIP的很多值得一提的结果，我总结起来如下所示：

- 它利用自然语言的监督信号，来训练一个比较好的视觉模型。一方面不需要再标注这些数据了；另一方面因为**训练的时候把图片和文字绑定在了一起**，所以训练的不再只是一个视觉特征了，是多模态特征，更容易进行zero-shot
- CLIP具有**出色的zero-shot迁移能力**。ImageNet上，甚至都没有使用标注数据来进行训练，CLIP就可以通过zero-shot分类实现与ResNet-50相当的性能表现。作者在 27 个数据集上衡量了 CLIP 做 zero-shot 迁移的效果，用zero-shot 的 CLIP 和linear probe的ResNet50对比（linear probe：把预训练好的模型中的参数冻结，只从里面去提取特征，然后只训练最后一层即 FC 分类头层）Zero-Shot CLIP 在其中16个数据集上都超过了 Linear Probe ResNet50
- 作者还进行了对抗样本训练，证明zero-shot的CLIP比之前只在imagenet上预训练的ResNet101模型效果好不少。

	Dataset Examples	ImageNet ResNet101	Zero-Shot CLIP	Δ Score
ImageNet		76.2	76.2	0%
ImageNetV2		64.3	70.1	+5.8%
ImageNet-R		37.7	88.9	+51.2%
ObjectNet		32.6	72.3	+39.7%
ImageNet Sketch		25.2	60.2	+35.0%
ImageNet-A		2.7	77.1	+74.4%

- Representation Learning领域，在27 个数据集上，CLIP**仅仅采用Linear Probe**效果就比所有的其他模型都要好；不只是zero-shot 和few-shot，用全部的数据去做训练时 CLIP 依然比其他模型强得多。这进一步说明了CLIP学习到的embedding具有良好的泛化性。

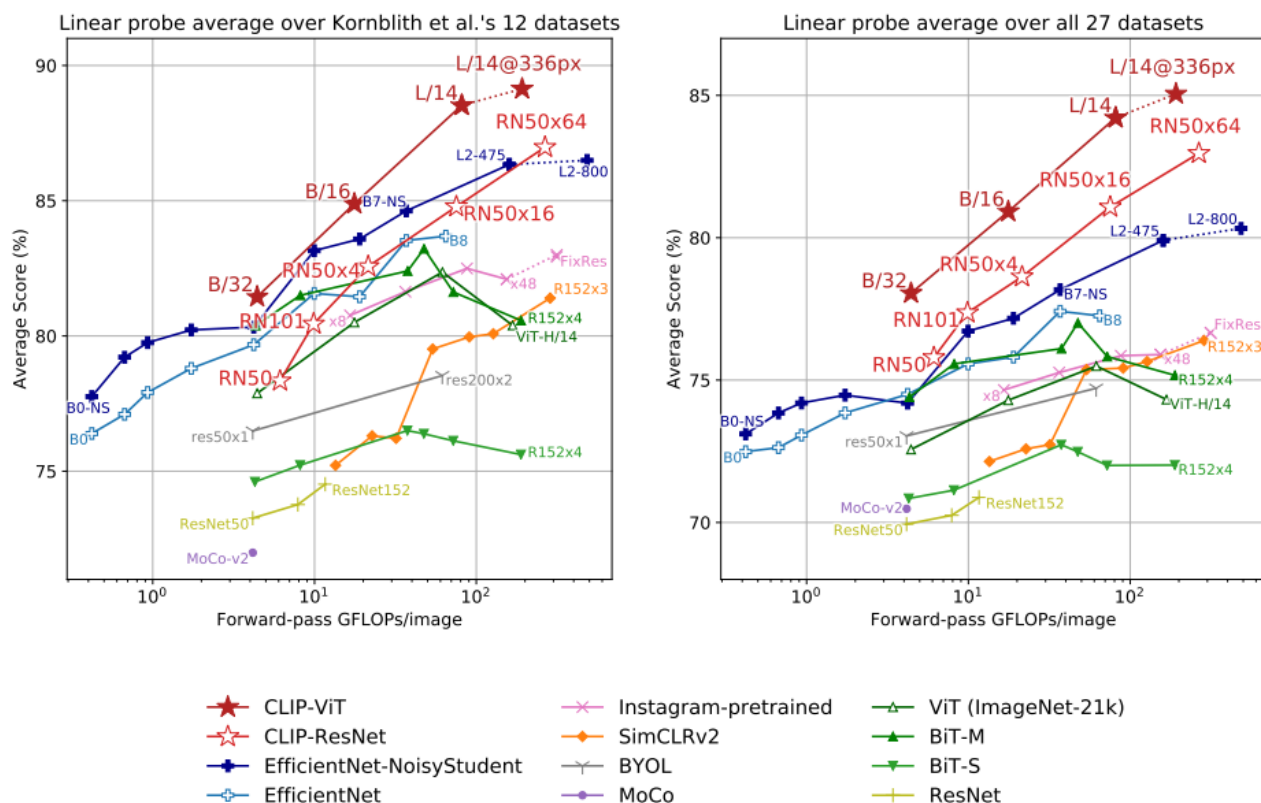


Figure 10. Linear probe performance of CLIP models in comparison with state-of-the-art computer vision models, including EfficientNet (Tan & Le, 2019; Xie et al., 2020), MoCo (Chen et al., 2020d), Instagram-pretrained ResNeXt models (Mahajan et al., 2018; Touvron et al., 2019), BiT (Kolesnikov et al., 2019), ViT (Dosovitskiy et al., 2020), SimCLRv2 (Chen et al., 2020c), BYOL (Grill et al., 2020), and the original ResNet models (He et al., 2016b). (Left) Scores are averaged over 12 datasets studied by Kornblith et al. (2019). (Right) Scores are averaged over 27 datasets that contain a wider variety of distributions. Dotted lines indicate models fine-tuned or evaluated on images at a higher-resolution than pre-training. See Table 10 for individual scores and Figure 20 for plots for each dataset.

CLIP也有一些不得不说的局限性如下所示：

- 在具有training splits的数据集上，zero-shot CLIP的性能和ResNet-50相持平。但在大多数数据集上，该基线的性能现在远低于总体水平。
- 在提高CLIP的任务学习和迁移能力方面还需要做大量的工作。估计zero-shot CLIP需要大约1000倍的计算能力才能达到最先进的整体性能。
- CLIP依赖于图像和文本之间的语义相似性，它可能无法区分一些细微的差别。例如“狗”和“狼”；“苹果”既可以指水果也可以指公司。
- CLIP的zero-shot性能在几种任务上仍然很弱：细粒度分类(如区分汽车模型、花卉种类和飞机变体)；无法处理抽象概念(如数数任务)；对于不太可能包含在CLIP预训练数据集集中的新任务（例如对照片中最近的汽车的距离进行分类），CLIP准确性甚至接近随机。

1.4.2 CLIP具体的结构

- CLIP的图像编码器可以采用ResNet-50作为基础模型，**但是也可以采用ViT**，我在本次实验中将官方训练的ViT-B/32，ViT-B/16都尝试了一下
- CLIP的文本编码器使用的是**Transformer**，它共有12层，512的隐层节点数以及8个头。

1.4.3 本次实验过程中可以使用CLIP吗？

我在网上看到的CLIP应用场景基本都类似这样——给定一张图片和一堆分类标签，比如cat, dog, bird，利用图片和文本编码器得到向量表示。然后分别计算这些标签与图片的余弦相似度；最终相似度最高的标签即是预测的分类结果。这个任务和我们本次实验中的任务比起来，不需要文本和图像之间有那么强的关联，换言之，这个任务中并没有怎么强调“模型融合”。那么，本次实验过程中我还能用CLIP这个思路吗？

我觉得这里是可以使用CLIP进行多模态分类任务的。在CLIP论文中所揭露的信息来看，CLIP学习到的embedding应该具备较强的迁移性，甚至只使用线性分类器就能取得不错的结果。如果我仅仅把CLIP作为特征提取器，分类的任务交给一个MLP去做。然后模型的输入是图像-文本pair，我们近似地把它们看作一个整体后embedding，最后只要对应地修改一下模型最后的线性层即可。这样理论上来说至少是可行的，但不一定是最优的，效果上理应会有一些的损失。

当然，我觉得我自行的解释可能并不足以消除你的疑惑，其实最开始我也不太理解为什么CLIP就真的能做到这一点。严格意义来说，图像文本匹配是我们最常见的图像-文本自监督预训练任务，这个任务考虑的是图文间内在特征的匹配程度，与共同考虑获得一个外在情感打分的任务并不一致。最有效的证明方式还是引用一下前人的工作。在这篇论文¹¹，作者介绍了一种关于社交网络的数据集——MVSA-Multiple，这个数据集与我们在这次实验中所采用的数据有极大的相似性——也是来源于社交网络的图像-文本pair，目标类别数也是3 (positive,neutral,negative)，就是规模上要大一些，三个类别加起来接近20000条。然后在此论文中，我们可以看到使用CLIP模型在MVSA-Multiple数据集也可以有不错的表现。

Features	MVSA-Single		MVSA-Multiple	
	ACC	F1	ACC	F1
E_T^{RAW}	68.46	66.01	62.83	57.70
E_T^{+HT}	68.88	66.49	60.09	55.58
E_T^{-HT}	67.50	64.50	65.65	59.61
C_T^{+HT}	71.00	68.47	58.52	54.50
C_T^{-HT}	68.72	65.59	65.70	59.43
E_o	64.69	61.40	65.28	56.63
E_s	64.66	61.51	65.13	56.00
E_a	64.89	61.65	64.85	56.02
E_f	59.63	48.48	66.41	53.21
C_I	72.09	70.03	65.42	59.22
$C_I + E_o + E_s + E_f$	70.29	69.51	63.65	59.87
$C_I + E_a + C_f$	69.70	68.66	63.79	60.33
$E_o + E_s + E_a + E_f$	66.42	65.40	63.49	58.58
$E_o + E_a + E_f$	66.10	65.05	63.75	58.89

总而言之，我认为CLIP能outstanding最本质的地方是在一个还不错的骨架网络情况下有足够量的数据参与训练，能work的核心原因在于预训练任务和本次实验中的下游任务也并没有那么大的差异。其实我对CLIP的代码之实现，也参考了一些这篇论文¹¹官方给出的代码¹²。

1.5 迁移学习的模式选择

迁移学习有两种常用的模式——**linear probe**和**fine tuning**。

它们一般都是和预训练搭配，是预训练模型适配下游任务时可选的训练方式。总结来看，它们的不同之处如下所示：

- fine tuning是使用预训练模型适配下游任务时，对整个预训练模型都进行参数更新。当然，微调也可以选择不进行全部更新，因为模型前几层提取到的信息一般比较共性，所以我們也可以在此时只对后面一部分模型进行更新
- linear probe在适配下游任务时，**干脆就把预训练模型给冻结了**，对它的参数几乎不更新，只对模型最后的线性层进行参数更新。
- linear probe**只对训练头进行训练**，不微调，节省了很多的时间，性能上有显著优势。
- 一般来说，线性探索一般用于检验预训练模型的好坏，但是结果上往往差于微调

结合1.3和1.4的理论基础，我的选择是这样的——对于resnet/RoBERTa模型，两种方式都可以试一试；但是CLIP类似于论文里描述的那样，我只尝试linear probe进行纯粹的静态特征抽取。

1.6 我选择的模型亮点阐述

其实我之前的理论介绍部分已经零零散散地说了一下自己选择的模型到底有什么亮点了，如果你要了解详情烦请阅读一下上面有点冗长的理论。我觉得自己这次实验最大的突破在于从理论入手，知道自己在干什么，而不是像之前那样有点被牵着鼻子走，自己都不太清楚自己具体在做什么。

- 使用linear probe，仅仅对模型最后的线性层进行参数更新，一方面大幅提升了训练的效率，另一方面也充分保证了模型的效果；
- 使用预训练模型，大幅减少了训练的耗时，我只需要让其适配对应的下游任务即可。并且，预训练模型的训练规模还是较充分的，可以得到一个较为通用的视觉-语言表示。
- 有充分的前人工作铺垫，模型整体来说较为成熟。比如说我使用的是encoder-only架构，不仅比使用encoder-decoder结构简单，而且性能被证实更好一些；模态之间cross-attention，比单独使用self-attention效果更好；ViT-based Patch Features在VLP任务中的效果多数情况来说要比CNN-based Grid Features更好。

2. 正式实验

在如此庞大的理论铺垫基础上，我终于正式展开了实验过程。讲真，如果没有那么强的理论支撑，我觉得自己这次实验可能也做不了很好。

2.1 数据预处理

数据预处理严格意义来说应该分为两个阶段——**数据的清理**和**特征提取**，我重点介绍的还是数据的清理。

2.1.1 文本清理

由于数据的不规则性，我本来想人工去实现相应的清理工作，但感觉这样的效果可能不如专业处理社交媒体文本的库好。我看到了一个轻量级的文本预处理工具**ekphrasis**，处理的对象是社交网络，比如twitter，facebook，该工具虽然有一些古老，但是在语言的规范化、意群分段、单词分割和拼写校正任务中还是可以起到不错的效果。

有了这个工具，参考它的标准用法，文本的处理就容易了许多，在这个过程中不要忘记表情的处理（这个直接打开文件可能看到的就是乱码）。值得注意的是，如果不进行清理，直接塞给CLIP，会报错。

```
from ekphrasis.classes.preprocessor import TextPreProcessor
from ekphrasis.classes.tokenizer import SocialTokenizer
from ekphrasis.dicts.emoticons import emoticons

text_processor = TextPreProcessor(
    # terms that will be normalized
    normalize=['url', 'email', 'percent', 'money', 'phone', 'user',
              'time', 'url', 'date', 'number'],
    # terms that will be annotated
    annotate={"hashtag", "allcaps", "elongated", "repeated",
            'emphasis', 'censored'},
    fix_html=True, # fix HTML tokens

    # corpus from which the word statistics are going to be used
    # for word segmentation
    segmenter="twitter",

    # corpus from which the word statistics are going to be used
    # for spell correction
    corrector="twitter",

    unpack_hashtags=True, # perform word segmentation on hashtags
    unpack_contractions=True, # Unpack contractions (can't -> can not)
    spell_correct_elong=False, # spell correction for elongated words

    # select a tokenizer. You can use SocialTokenizer, or pass your own
    # the tokenizer, should take as input a string and return a list of
    tokens
    tokenizer=SocialTokenizer(lowercase=True).tokenize,
```

```

# list of dictionaries, for replacing tokens extracted from the text,
# with other expressions. You can pass more than one dictionaries.
dicts=[emojis]

)

def text_transform(tweet):
    proc_tweet = text_processor.pre_process_doc(tweet)
    clean_tweet = [word.strip() for word in proc_tweet if not re.search(r"
[^\a-z0-9.\s]+", word)]
    clean_tweet = [word for word in clean_tweet if word not in ['rt', 'http',
'https', 'htt']]

    return " ".join(clean_tweet)

```

经过这个工具处理过后的文本，只需要利用相应的tokenizer分词即可。

2.1.2 图像清理

说实话其实图像的清理要比文本容易很多，甚至clip都不需要什么清理过程。我们只需要写一个模板，然后把交给ResNet的图像经由这个模板处理即可：

```

img_transforms = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

```

- 图像调整大小为256x256像素（确保了所有的图像在输入模型之前具有相同的尺寸）
- 从调整大小后的图像中心裁剪出224x224大小的区域，这个操作用于获取图像的主要内容
- 图像数据转换为PyTorch张量，然后对图像张量进行标准化处理，将图像的像素值进行归一化操作，这个操作有助于提高模型的训练速度和准确性，同时有助于避免梯度爆炸或梯度消失的问题。
- 为什么最后我将图像的像素值进行归一化操作，设置其均值为0.485、0.456、0.406，标准差为0.229、0.224、0.225？这个主要也是参考pytorch官方实现，详细的解释可以看这里。 13

2.1.3 清理之后的特征提取

- 视觉部分特征提取器是ResNet50 文本部分特征提取器是RoBERTa
- CLIP有单独的特征提取器。这个特征提取器调用官方的img_preprocess就能对图像进行一些清理工作。具体的分类任务交给一个MLP来做

- ResNet50: 通过全部模型作为图像embedding; RoBERTa: 仅使用经过编码器的池化输出 **pooler_output**作为文本embedding; CLIP: 通过全部模型作为图像和文本的embedding

2.1.4 预处理之后的数据如何调用？

经过2.1.1和2.1.2的步骤，我得到了各模型所对应的特征信息。值得注意的是因为模型不同，最后生成的特征维度不同，这个在之后训练模型时我们应该怎样调用呢？

其实只要明确维度问题，在之后不同模型时写好对应的代码就很好办。我在此专门检查了一下不同特征的embedding维度：

模型	维度
ResNet	1000
RoBERTa	768
CLIP	512

2.2 训练及预测

本次训练时，我不再像第四次实验那样，直接使用 Huggingface的Trainer进行训练了——虽然那样确实可以简化很多工作，但是因为封装地太完全了，调试过程会十分痛苦以至于我最后可能不得不因此而放弃一些更高级的功能。所以，我本次实验还是老老实实用torch来实现训练及预测过程。

为了介绍训练过程，**我也没必要每行代码都粘贴到报告里**，所以这里只筛选一些我觉得重要的点来谈。广义层面这一部分我要来谈数据预处理之后的各项工作，而不仅是介绍训练的各种问题。

2.2.1 模型框架

在训练之前肯定要架构模型的框架。模型如果要分类的话可以分为两类——一类是单模态的；另一类是多模态融合的。在设计模型时，我也采用了dropout方法来降低过拟合程度（dropout率0.3）采用BatchNorm来增强模型泛化性（毕竟输入的都是embedding好的，所以这里是1d）我的分类头只设置了一层，整体结构比较简易

单模态的模型十分简单，其中分类头大小是D

```
class single(nn.Module):
    def __init__(self, dim, D):
        super(single, self).__init__()

        self.fc = nn.Linear(dim, D)
        self.output = nn.Linear(D, 3)

        self.bn = nn.BatchNorm1d(D)
        self.drop = nn.Dropout(0.3)
```

```

self.relu = nn.ReLU()

def forward(self, emb):
    out = self.drop(self.relu(self.bn(self.fc(emb))))
    return self.output(out)

```

多模态的话如果是直接简单拼接，其实和单模态也是类似的，注意一下拼接那里就行。在拼接之前要先拓展到相同的维度，保证两个通道可以进行拼接。

```

class fusion(nn.Module):
    def __init__(self, vdim, tdim, D):
        super(fusion, self).__init__()

        self.vfc = nn.Linear(vdim, D)
        self.tfc = nn.Linear(tdim, D)
        self.output = nn.Linear(2 * D, 3)

        self.bn1 = nn.BatchNorm1d(D)
        self.bn2 = nn.BatchNorm1d(D)
        self.drop1 = nn.Dropout(0.3)
        self.drop2 = nn.Dropout(0.3)
        self.relu = nn.ReLU()

    def forward(self, v_emb, t_emb):
        v_emb = self.drop1(self.relu(self.bn1(self.vfc(v_emb))))
        t_emb = self.drop2(self.relu(self.bn2(self.tfc(t_emb))))
        fusion = torch.cat((v_emb, t_emb), axis=1)
        return self.output(fusion)

```

多模态的话如果要采用交叉注意力机制，就更复杂了一些。fusionWithCrossAttention引入了一个多头注意力机制，首先生成对应的key,value,query，之后用torch.stack把张量叠起来，再用多头注意力机制处理，处理之后别忘了复原到原始的维度。

```

class fusionwithCrossAttention(nn.Module):
    def __init__(self, vdim, tdim, D):
        super(fusionwithCrossAttention, self).__init__()
        self.D = D

        self.vfc = nn.Linear(vdim, D)
        self.tfc = nn.Linear(tdim, D)
        self.output = nn.Linear(2 * D, 3)

        self.multihead_attn = nn.MultiheadAttention(embed_dim=D, num_heads=8,
batch_first=True)
        self.tk = nn.Linear(D, D)

```



```

self.tv = nn.Linear(D, D)
self.tq = nn.Linear(D, D)
self.vk = nn.Linear(D, D)
self.vv = nn.Linear(D, D)
self.vq = nn.Linear(D, D)

self.flat = nn.Flatten()
self.bn1 = nn.BatchNorm1d(D)
self.bn2 = nn.BatchNorm1d(D)
self.drop1 = nn.Dropout(0.3)
self.drop2 = nn.Dropout(0.3)
self.relu = nn.ReLU()

def forward(self, v_emb, t_emb):
    v_emb = self.drop1(self.relu(self.bn1(self.vfc(v_emb))))
    t_emb = self.drop2(self.relu(self.bn2(self.tfc(t_emb))))

    tk = self.tk(t_emb)
    tv = self.tv(t_emb)
    tq = self.tq(t_emb)
    vk = self.vk(v_emb)
    vv = self.vv(v_emb)
    vq = self.vq(v_emb)
    key = torch.stack((tk, vk), dim=1)
    value = torch.stack((tv, vv), dim=1)
    query = torch.stack((tq, vq), dim=1)
    fusion, _ = self.multihead_attn(query, key, value)
    fusion = fusion.reshape(-1, 2 * self.D)

    return self.output(fusion)

```

以上这两类模型面向的其实进行的是linear probe训练方法，如果要进行fine tuning,模型的实现其实有一些不同（毕竟调用预训练的resnet和roberta有一些不同的参数），但是整体架构是类似的，具体的详见代码。最后我在探索了一番之后发现dropout和BN组装在一起效果并不是很好，于是删去了dropout，具体详见报告3.7部分。

2.2.2 训练

0. 因为我两种迁移学习的模式都尝试了，它们在训练过程中其实存在一些差异，但又不想整理那么复杂的代码了，所以两者的逻辑分别在train_linear_probe.py和train_fine_tuning.py中
1. 选择的模型不同，衔接预处理工作，要调整对应的图像、文本输入维度

```

if hyper_dict.vtype == "clip":
    vdim = 512
elif hyper_dict.vtype == "resnet":
    vdim = 1000

if hyper_dict.ttype == "clip":
    tdim = 512
elif hyper_dict.ttype == "roberta":
    tdim = 768

```

2. 设置fp16,将模型参数和梯度存储为 16 位浮点数,而不是标准的 32 位的,减少显存使用、加速计算来提高训练速度 `scaler = torch.cuda.amp.GradScaler(enabled=use_amp)` 之后,在训练过程中对scaler如常backward, step, update即可。开启16位浮点数之后, 要使用 Gradient scaling防止在反向传播过程由于中梯度太小 (float16无法表示小幅值的变化) 从而下溢为0的情况。而且别忘了手动开启混合精度。

3. 训练过程的可复现性。

自己之前也一直没太在意这一块内容, 所以之前的四次我都没有有意识设置随机种子保障可复现性的过程。

```

def set_seed(seed):
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    np.random.seed(seed)
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    set_seed(RANDOM_SEED)

```

为了保险起见, 我还加了两句话:

```

torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

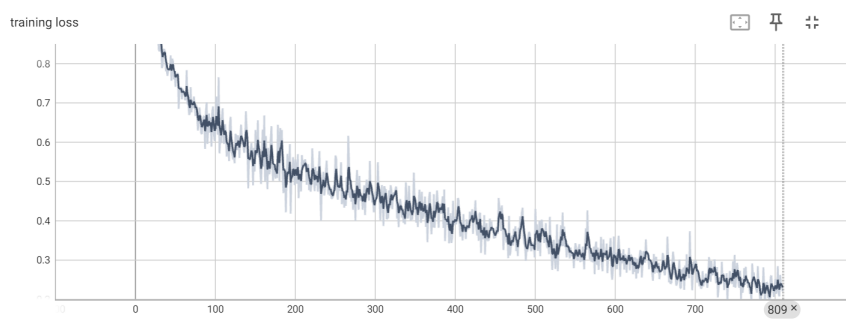
```

4. 优化器我统一选择的是Adam, 损失函数则采用的是之前实验通用的交叉熵。
5. 在模型每一步训练的过程中, 要注意输入会因为是否融合而有所变化。
6. linear probe和fine tuning的训练代码其实差不多, 核心的差异在于后者会更新预训练模型的特征提取器而前者并不会, 所以fine tuning增添了对input_ids和attention_mask的处理。因为要考虑这两者了, 所以两种迁移学习方式所用的自定义Dataset类也有所不同。具体的训练过程请见代码。
7. 训练之后的验证简直就是训练过程的翻版, 注意输入的data有差异! 而且别忘了此时模型要转换模式了!! (eval不能忘了)

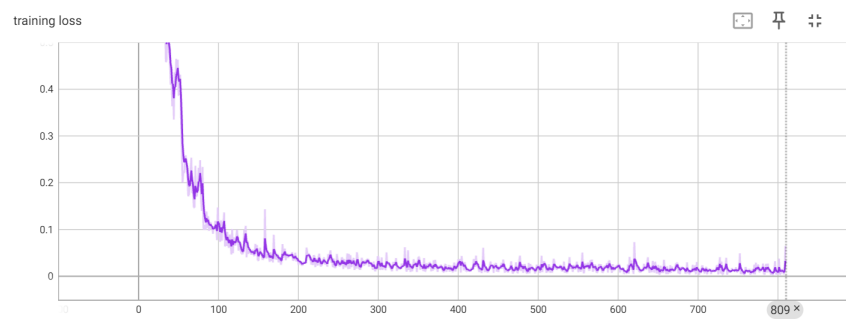
8. 训练的详细过程，我不再像之前的实验那样傻傻的搞个列表-存入列表-交给matplotlib画图了。这次我直接使用tensorboard的**SummaryWriter**，简单而又明确地展现可视化数据
9. 有一点差点忘记说了，就是我划分训练集和验证集的比例默认是85:15,这主要是为了让模型充分地得到训练。本来我想设置为9:1的，没有如此痛下杀手的原因在于感觉数据量有点小（才4000条）。需要注意的是训练数据本身其实类别上本身是比较失衡的，我还进行了一点特别训练——即随机cut掉数据集中特别多的postive样本组成一个新的数据集，再拿此去和原始的进行对比，具体的在之后结果部分有展开。

我尝试着调试了很多次，中间出现了一些因为自己设置的问题导致损失函数无法收敛的情况。经过一番修正，我最终让模型的训练过程收敛了（损失函数角度）。由于过程过于冗长，我就筛选几个例子来介绍一下吧。

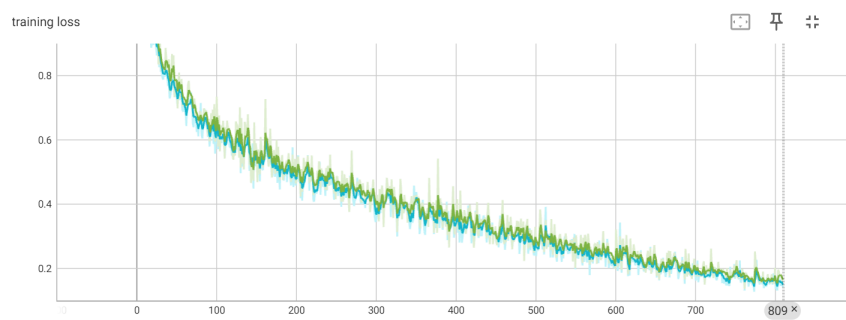
linear probe+resnet/roberta D=128 lr=1e-4 epochs=30 batchsize=128



fine tuning+resnet/roberta D=128 lr=1e-4 epochs=30 batchsize=128

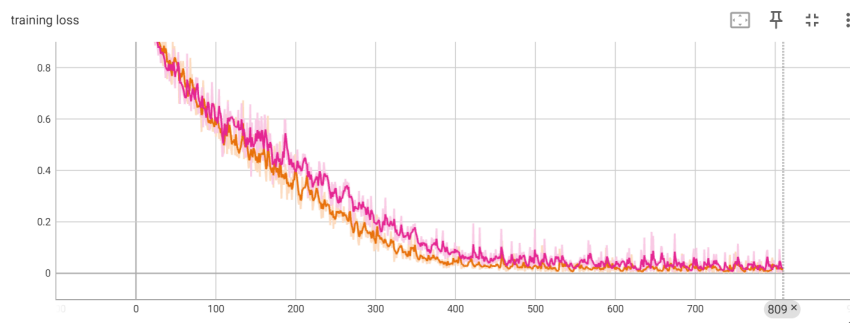


linear probe+clip(ViT-B/16 和 ViT-B/32) D=128 lr=1e-4 epochs=30 batchsize=128



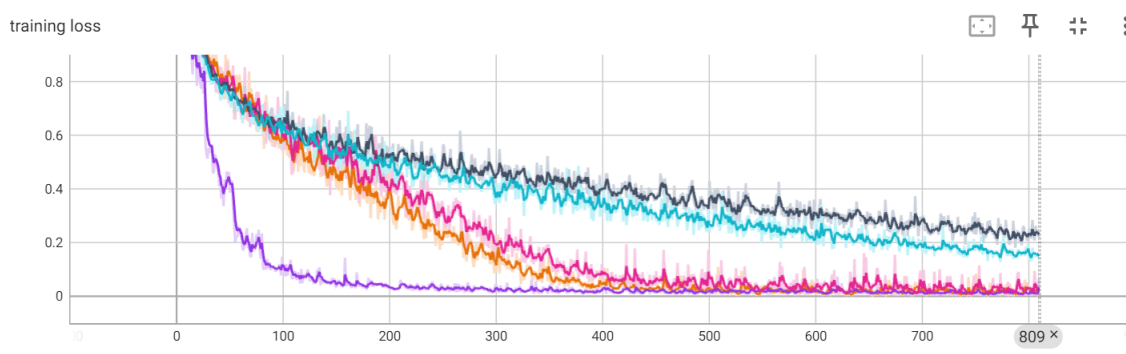
其中蓝线是ViT-B/16 绿线是ViT-B/32 可以看到两者的有着高度的相似性。

如果采用交叉注意力机制，linear probe D=128 lr=1e-4 epochs=30 batchsize=128的情况如下图所示：



其中红线是resnet/roberta，黄线是CLIP

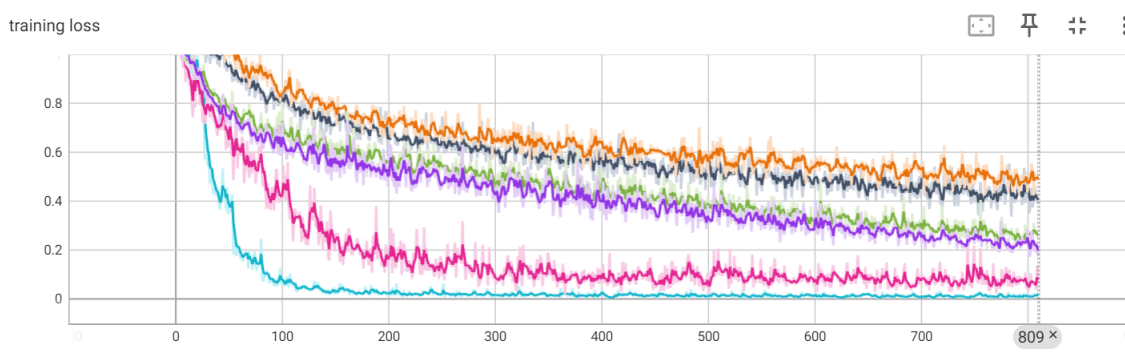
将几张重要的曲线放在一起做对比：



图像的颜色与之前分别介绍时候一致，这里就不重复说了。可以发现：

- 采用fine tuning,训练收敛的速度要比linear probe快得多
- 采用交叉注意力机制比直接拼接无论是从收敛情况还是模型的效果来看都要更好一些，这也为我之后的选择埋下了一点伏笔。

在进行消融实验时，我的训练损失函数大概是这样的：



在这张图中，紫线代表clip仅图像+linear probe，绿线代表clip仅文字+linear probe，黄线代表resnet仅图像+linear probe，黑线代表roberta仅文字+linear probe，蓝线代表resnet仅图像+fine tuning，红线代表roberta仅文字+fine tuning。其实这张图片中所反映的规律与我刚刚总结的几乎一致。

2.2.3 预测

其实预测过程没什么可以讨论的了，因为和训练过程还是比较相似的，本质差异还是在输入数据集的不同。

与此同时，我为了构造DataLoader而自行构建的Dataset类也有所不同——因为训练过程的数据都是有标签的，但是预测过程输入时是没有标签的。

3. 碰到的问题及解决

3.1 AttributeError: module 'clip' has no attribute 'load'

不能直接 `pip install clip` 而应该先 `pip install ftfy regex tqdm` 再 `pip install git+https://github.com/openai/CLIP.git`

3.2 RuntimeError: "nll_loss_forward_reduce_cuda_kernel_2d_index" not implemented for 'Int'

这个主要是数据类型的问题，交给cuda的整数一般是64位的，这个在dataset中把int32修改为int64问题就解决了。

3.3 连接云主机的Tensorboard

因为我的代码基本都是在云主机上运行的（自己的电脑太渣了~），而且运行日志实在是太多了，没必要去一个一个下载，所以要查阅TensorBoard中的报告就需要连接到云主机。但是，我开了一个runs（log目录）下的notebook,然后输入 `!tensorboard --logdir=./` 输出也确实没问题，云主机的6006端口运行着Tensorboard服务，但是因为没广播，即便我云主机安全设置把所有的端口都解锁了，我的主机依旧无法访问xxxx:6006(提示拒绝访问)。后来我在网上看到一条方法，就是采用一个类似端口映射的方法，在我的主机上输入 `ssh -N -f -L`

`localhost:16006:localhost:6006 ubuntu@xxxx(云主机地址)` 之后，我再在本地的16006端口就能访问tensorboard了

3.4 HuggingFace无法访问 Connection error

这个中国特色的问题其实我在实验四里也遇到了，解决方法与之一样，就是先在连接外网的情况下把对应的tokenizer和model下载下来，之后从本地读取即可。这两件东西我现在本地有外网的前提下下载了，然后通过scp上传到了服务器。

我其实很好奇为何从openai加载clip就没碰到网络的问题/疑惑.....

3.5 Value Error: not enough values to unpack

这个问题之所以出现，还是因为自己设置的Dataset类在不同学习方式时有所不同但自己忘记调整这一点所导致的。比如说，训练过程的数据都是有标签的，但是预测过程输入时是没有标签的。不得不说，我在本次实验里设置了好多种MyDataset，每个都因为交给DataLoader处理的方式有所差异而显得不尽相同。

3.6 CLIP通道输入错误导致损失函数很大

在调试时，由于没有特别注明，我错误地把图像输入到了CLIP的文字通道中，结果程序还真的能跑起来（因为维度是对的）但是训练时损失函数很大，看起来几乎无法收敛。经过一番检查，我才发现自己通道搞混了。

3.7 使用已经不那么流行的dropout机制是不是不太好？

我在网上看到了这篇博文¹⁴，这篇文章提到目前在正则化领域dropout已经不是那么风靡了，同时还通过它和Batch Normalization的对比实验证实了BN在准确率和损失率上表现要优于Dropout。事实真的是如此吗？我在网上也查阅了一些相关的资料，在这篇论文中¹⁵，有一个结论便是加了Batch Normalization之后，再加dropout，模型性能反而更差了，**当 Dropout 和 BN 结合在一起时并不能获得额外的增益。**

同样的参数，假如我将dropout删去，训练时在验证集上的准确率却出现了略微程度的增幅。在网上看了一些讲解后我发现NLP领域BN好像和CV领域存在着一些差异，甚至有论文研究为何Transformer不使用BN¹⁶，具体的机制我并不是特别理解，但最终为了保险起见我还是删去了dropout。

4. 结果&分析

实验结果分为几个部分，一部分是多模态融合模型在验证集上的结果，另一部分是消融实验的结果（即分别只输入文本或图像数据，我的多模态融合模型在验证集会获得怎样的表现。）还有一部分是我自己的小实验（随机cut掉数据集中特别多的postive样本组成一个新的数据集，再拿此去和原始的进行对比）。

4.1 多模态融合模型在验证集上的结果

epochs默认为30 batchsize默认为128 这两个超参数我就不调整了。我主要调整两个超参数——学习率lr和分类头大小D

resnet50+roberta这个多模态融合模型在验证集上结果如下所示：

学习方式	分类头大小D	学习率lr	最佳准确率
linear probe	128	1e-4	73.5%
linear probe	128	1e-5	71.8%
linear probe	256	1e-4	72.5%
linear probe	256	1e-5	72.3%
fine tuning	128	1e-4	67.5%
fine tuning	128	1e-5	64.3%
fine tuning	256	1e-4	68.7%
fine tuning	256	1e-5	64.2%

CLIP采用linear probe，在验证集上结果如下所示：

Vision Encoder	分类头大小D	学习率lr	最佳准确率
ViT-B/16	128	1e-4	78.5%
ViT-B/16	128	1e-5	74.8%
ViT-B/16	256	1e-4	78.7%
ViT-B/16	256	1e-5	76.2%
ViT-B/32	128	1e-4	78.5%
ViT-B/32	128	1e-5	73.7%
ViT-B/32	256	1e-4	77.5%
ViT-B/32	256	1e-5	75.0%

可以看到ViT-B/32和ViT-B/16并没有什么实质性的差距。看来，增大patch size对于这个不算太大的数据集来说并没有什么显著的影响，我最终采用的是ViT-B/16

结合实验报告前述的理论知识，我们知道模型融合有两种方式——直接拼接和交叉注意力；前面的结果都来自于直接拼接。那么假如使用交叉注意力来融合，会有什么差异呢？我以linear probe为例来测试了一下：

模型	分类头大小D	学习率lr	最佳准确率
resnet50+roberta	128	1e-4	72.8%
resnet50+roberta	128	1e-5	72.2%
resnet50+roberta	256	1e-4	72.0%
resnet50+roberta	256	1e-5	74.3%
CLIP	128	1e-4	76.7%
CLIP	128	1e-5	74.8%
CLIP	256	1e-4	77.0%
CLIP	256	1e-5	76.7%

对比可以发现，两种不同的模型融合方式在此实验中并不存在什么显著的差异，但是结合之前训练过程时对于损失函数值变化的观察，我还是在之后的实验中统一采用交叉注意力机制。

4.2 消融实验的结果

epochs默认为30 batchsize默认为128 这里的D和lr分别都是上面的实验中选择的最佳超参

Vision Encoder	Text Encoder	迁移模式	D	lr	最佳准确率
clip	NONE	linear probe	256	1e-4	76.2%
NONE	clip	linear probe	256	1e-4	73.2%
resnet50	NONE	linear probe	128	1e-4	68.5%
NONE	roberta	linear probe	128	1e-4	70.3%
resnet50	NONE	fine tuning	256	1e-4	66.3%
NONE	roberta	fine tuning	256	1e-4	72.7%

4.3 我的小实验结果

我随机cut掉数据集中特别多的postive样本，使其从2388条下降到了1188条（cut之后数据总量为2800，要不然train_test_split会报错），与negative数量相近，组成了一个新的数据集，然后在此训练之后又会有什么发现呢？我使用clip来探索了一下。

模式	分类头大小D	学习率lr	最佳准确率
原始数据集	128	1e-4	76.7%

模式	分类头大小D	学习率lr	最佳准确率
原始数据集	256	1e-4	77.0%
cut后的数据集	128	1e-4	72.1%
cut后的数据集	256	1e-4	71.9%

可以看到，随机cut掉了过多的positive类别之后，CLIP的最佳准确率并没有出现显著的降幅，出现这个情况的主要原因还是因为CLIP采用的linear probe方法主要训练的是最后的线性层。就我的理解来看，出现下降的主要原因还是数据规模变小了，模型的训练充分性可能并不是那么大。这证明了采用CLIP方法的话不用特别需要担心数据集的类别失衡问题。

4.4 结果分析

总而言之，通过分析不同情况下两大类模型的表现，我发现CLIP都取得了还不错的结果，整体表现比resnet50+roberta要好一些，至于具体的对比请看报告的理论部分。但是整体来说，三种预训练模型都取得了不错的表现。当然，我这里因为没设置baseline模型，所以也不好评价使用预训练模型相对于没有使用到底有多大的提升。

在我的消融实验中，通过单模态与多模态的对比，我发现多模态时效果确实要比单模态好一些，虽然好的程度并不算很多，至于为什么会好，理论部分也有阐述的，简而言之就是单模态对于信息的提取都是局部的，彼此之间需要一些结合才能形成完整的认知。clip与resnet50+roberta展现了不同的情况——前者image only的效果比text only更好一些；后者则反之。我猜测出现这种颠倒情况的核心原因在于它们两类的架构就不太相同，即使Text Encoder都是Transformer类架构，但是Vision Encoder部分前者我使用的是更先进的ViT而后者还是比较传统的CNN结构。

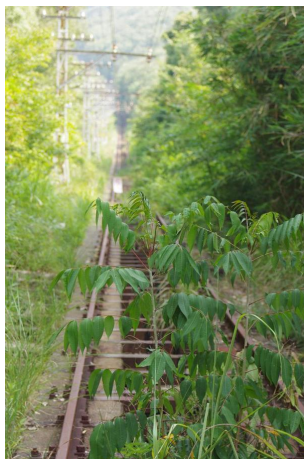
4.5 误判问题

经过上述的实验，我最终选择的是CLIP D=256 lr=1e-4 当然为了稳妥起见我还用相同的超参数fine tuning方式训练了一套resnet50+roberta融合模型，生成的答案在answer1.txt中

我在最后检查了一下这个模型，让它在训练数据上来跑一下，看看模型的误判率如何。结果是4000条数据总计误判164条，误判率4.1%。整体来说，我觉得这个结果还是可以接受的。但是抱着试一试的心态我也看了一下误判到底出现在什么位置。

positive误判为了negative共计47条

以2020.jpg和2020.txt来看



#廃墟 #廃線 #abandoned #写真撮ってる人と繋がりたい #写真好きな人と繋がりたい

其实刚看到这个风景，通过 #废墟 这个标签，我自己也很容易把它分类为消极的

以2965.jpg和2965.txt来看：



Took a photo in Colorado on a foggy morning. via /r/pics #pics

这个雾霭弥漫，虽然这幅图也许是积极的，但我觉得这种阴郁的氛围很难让人感觉到positive

positive误判为了neutral共计18条；negative误判为了positive共计40；negative误判为了neutral共计10条

neutral误判为了positive共计25条

以3678.jpg和3678.txt来看：



RT @ACunninghamMP: Storming opening speech from @Andy4Leader speaking in person in Teesside on why he should be Labour Party leader

neutral误判为了negative共计24条

以2658.jpg和2658.txt来看：



why are you feeling despondent? Take the quiz:

总而言之，我觉得如果图像上有文字或者图像与文字需要比较强的先验知识的情况下，模型就非常容易发生误判。前者尚且还可以通过图像文字扫描技术将图像中的文字提取出来之后再去训练（当然这样可能破坏文字的位置等信息），但是后者的完善只能靠模型庞大的预训练了，现在的大模型被预训练得已经达到一个又一个领域的“专家”水平了。

5. 总结&感想

这应该是自己做过的最认真的实验了，前前后后花了不少的时间。老师们一直在强调“你们学习理论很重要，但是更重要的是学习理论后的动手实践”，这个观点是没问题的，但现实中却出现了很多问题——如此多的内容，《当代人工智能》的理论课时又那么少，上课时老师只能入个门告诉我们有x知识点。要想真的理解并掌握相关的知识，离不开课下海量的功夫。本次实验中，我主要花费的功夫其实是在补足理论层面的短板。在全面回顾了一学期所学知识的前提下，透过前人的经典论文我知道了多模态分类任务的基础架构，我了解了VLP工作的各个组件，使用预训练模型构建了resnet50+roberta为基础的基础模型，并且更进一步地采用CLIP架构进行实验、做出对比。

回顾起来，自己这一学期学习人工智能的历程还是比较曲折的。最开始懵懵懂懂，啥都不会；后来伴随着上课、上网查阅资料终于学到了很多。在12月中下旬，伴随着自己对Transformer架构乃至预训练模型的不甚理解，其他科目海量的作业淹没了我、期末月庞大的复习压力吞噬了我，我压根没有多少功夫静下心来去学。“不是在忙，就是在忙的路上”，以至于做出了自我感觉最差的第四次实验/捂脸。所幸第五次实验在期末之后，自己终于有时间完成一整套学习理论-实验的流程。

在做完这个实验后，我发现自己对NLP，CV，多模态等领域的基础知识和实验都有了一个较好的认识，这为我之后的学习无疑打下了坚实的基础。现在的我才是一个大二上（不是大三上）刚刚结束的学生，之后课业的压力应该没那么艰巨了，希望自己利用好这充分的时间，利用好这一学期学到的知识，在AI的领域砥砺前行吧！

6. 引用&参考

1. Transformer架构的整体指南: <https://luxiangdong.com/2023/09/10/trans/#/> [↵](#)
2. Zi-Yi Dou, et al. An Empirical Study of Training End-to-End Vision-and-Language Transformers [J]. arXiv preprint arXiv: 2111.02387 [↵](#)
3. Dosovitskiy, Alexey, et al. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929 (2020). [↵](#)
4. 狗都能看懂的Vision Transformer的讲解和代码实现vision transformer代码-CSDN博客: https://blog.csdn.net/weixin_42392454/article/details/122667271 [↵](#)
5. ViT论文逐段精读【论文精读】: <https://www.youtube.com/watch?v=FRFt3x0bO94> [↵](#)
6. Kaiming He, et al: Deep residual learning for image recognition, 2016. arXiv:1512.03385 [↵](#)
7. Liu, Yinhan, et al. Roberta: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692. [↵](#)
8. 论文精读: <https://www.youtube.com/playlist?list=PLFXJ6jwg0qW-7UM8iUTj3qKqdhbQULP5I> [↵](#)
9. CLIP 论文逐段精读【论文精读】: https://www.youtube.com/watch?v=OZF1t_Hieq8 [↵](#)
10. Alec Radford, et al. Learning transferable visual models from natural language supervision. International Conference on Machine Learning. PMLR, 2021. arXiv:2103.00020 [↵](#)
11. Cheema, Gullal S., et al. A fair and comprehensive comparison of multimodal tweet sentiment analysis methods [J]. Proceedings of the 2021 Workshop on Multi-Modal Pre-Training for Multimedia Understanding. 2021. [↵](#) [↵](#)
12. https://github.com/cleopatra-itn/fair_multimodal_sentiment [↵](#)
13. Why Pytorch officially use mean=[0.485, 0.456, 0.406] and std=[0.229, 0.224, 0.225] to normalize images?: <https://stackoverflow.com/questions/58151507/why-pytorch-officially-use-mean-0-485-0-456-0-406-and-std-0-229-0-224-0-2> [↵](#)
14. 深度学习基础系列（九） | Dropout VS Batch Normalization? 是时候放弃Dropout了 - 可可心心 - 博客园: <https://www.cnblogs.com/hutao722/p/9946047.html> [↵](#)
15. Sergey Ioffe, et al. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. arXiv:1502.03167 [↵](#)
16. Sheng Shen, et al. PowerNorm: Rethinking Batch Normalization in Transformers. arXiv: 2003.07845 [↵](#)